

ファイル構成・依存関係マップ（レンタルスタジオサーチアプリ）

本書は「どの画面が何をするか」「どのファイルがどのファイルを読んでいるか」を一覧できるようにするためのリファレンス。釣行AIアプリ（fishing-ai-app）の同名資料をベースに、ドメイン変換（Spots→Studios等）と、スコアリング刷新（口コミ・設備・最寄り駅距離・収容人数・料金）を反映した最新版。仕様の背景・意思決定理由はAWS構成・サービス連携.html／詳細設計.html／企画・設計アーカイブ.htmlを参照。

1. 画面一覧（フロントエンド）

URL	画面名	ページファイル	やること
/	おすすめ	pages/TopPage.tsx	AIスコア順のTOP3・一覧表示、AI分析実行、現在地から新スタジオ探索、現在地基準の再ランキング
/map	地図	pages/MapPage.tsx	Google Maps上に全スタジオをスコア色分けピン表示
/studios	スタジオ	pages/StudiosPage.tsx	全スタジオ一覧（スコア・設備・ナビリンク）、キーワード・設備タグでの絞り込み
/favorites	保存済み	pages/FavoritesPage.tsx	お気に入り登録済みスタジオのみ表示
/posts	レビュー	pages/PostsPage.tsx	レビュー投稿（★評価・本文・写真）の一覧・作成・編集・削除。編集は本人の投稿のみ、投稿先スタジオは変更不可
/chat	AI相談	pages/ChatPage.tsx	写真添付対応のAIチャット、会話履歴、送信済みメッセージの編集。実データ（Studios/Recommendations）に基づいた回答
/guide	使い方	pages/GuidePage.tsx	画面ごとの使い方・ホーム画面への追加手順・FAQ

ルーティング定義は App.tsx。全ページ共通のヘッダーは components/NavBar.tsx（ロゴクリックで / に戻る、モバイルではラベル非表示、右端にログイン状態を表示）。

ログイン不要（閲覧）：おすすめ・地図・スタジオ・レビュー投稿の一覧表示。

ログイン必須（操作）：お気に入り登録/削除、投稿の作成/編集/削除、AI相談、AI分析実行、現在地から新スタジオ探索（Places/Claude APIのコスト保護のため）。未ログインでこれらを実行しようとするとCognito Hosted UI（Googleログイン画面）へ自動遷移するか（お気に入り・投稿作成・AI分析実行・新スタジオ探索）、ページ自体がログイン案内を表示する（AI相談）。

TOP3カード（RecommendationCard.tsx）はスタジオ写真が登録されている場合、カード上部にヒーロー写真を表示する（その他一覧・お気に入り一覧はホバー/長押しプレビューのみ）。釣行AIアプリの釣具店検索に相当する機能は本アプリには無い（v1では対象外）。

2. フロントエンド ファイル依存関係

2.1 ページ → フック → コンポーネント

ページ	使用するフック	使用する主なコンポーネント
-----	---------	---------------

TopPage.tsx	useRecommendations, useFavorites	RecommendationCard, AiBatchButton, StudioDiscoveryButton, DetailModal, NearbyModal
MapPage.tsx	(なし。api/client を直接呼ぶ)	@vis.gl/react-google-maps (外部ライブラリ)
StudiosPage.tsx	(なし。api/client を直接呼ぶ)	ImagePreviewPopover
FavoritesPage.tsx	useFavorites, useRecommendations	RecommendationCard, DetailModal
PostsPage.tsx	usePosts	PostForm
ChatPage.tsx	useChat	ChatInput, ChatHistoryPanel
GuidePage.tsx	(なし。静的コンテンツのみ)	(なし)

2.2 コンポーネント間の依存

コンポーネント	使われる場所	内部で使うもの
RecommendationCard.tsx	TopPage, FavoritesPage, NearbyModal	utils/score.ts, ImagePreviewPopover
DetailModal.tsx	TopPage, FavoritesPage, NearbyModal	utils/score.ts, ImagePreviewPopover, api/client (写真アップロード)
NearbyModal.tsx	TopPage	hooks/useGeolocation, utils/distance.ts, utils/score.ts, RecommendationCard
ImagePreviewPopover.tsx	RecommendationCard, StudiosPage, DetailModal	hooks/useLongPress (imageUrlのみ表示。釣行AIアプリのWikipedia画像フォールバックは対象外のため無し)
PostForm.tsx	PostsPage	api/client (署名付きアップロード・投稿作成)。★評価入力UIを含む
ChatInput.tsx / ChatHistoryPanel.tsx	ChatPage	(propsで受けたコールバックのみ) ChatInputは hooks/useGeolocationで現在地共有トグルを実装
AiBatchButton.tsx / StudioDiscoveryButton.tsx	TopPage	StudioDiscoveryButtonのみ api/client (runStudioDiscovery) を直接呼ぶ
ProfileModal.tsx	NavBar	(propsで受けたコールバックのみ。API直呼びなし)

2.3 フック → APIクライアント

フック	呼び出す api/client.ts の関数
useRecommendations.ts	getRecommendations, runAiBatch (未ログイン時はrunAiBatch実行前にsignInRedirect)
useFavorites.ts	getFavorites, addFavorite, removeFavorite (未ログイン時はfetchせず空配列、トグル時はsignInRedirect)
usePosts.ts	getPosts, createPost, updatePost, deletePost (削除失敗時はdeleteErrorに反映)
useChat.ts	sendChatMessage, editChatMessage, getChatHistory, getChat, deleteChat, getPresignedUploadUrl, uploadImageToS3 (429応答はレート制限メッセージとして表示。lat/lngを渡せる)
useProfile.ts	getMyProfile, updateMyProfile (未ログイン時はfetchしない。NavBarから使われる)
useGeolocation.ts	(なし。navigator.geolocation をラップするのみ。TopPage/StudioDiscoveryButton/ChatInputから使われる)

useLongPress.ts	(なし。hover/長押し検知のみのUIフック)
-----------------	--------------------------

api/client.ts が全エンドポイント呼び出しを一手に引き受ける唯一の窓口（共有axiosインスタンス）。釣行AIアプリにあった魚種画像取得用の api/wikipedia.ts は、対応する「設備画像」の概念が無いため本アプリには存在しない。

2.4 認証まわりのファイル

ファイル	役割
auth/authConfig.ts	react-oidc-context の AuthProvider に渡す設定（authority/client_id/redirect_uri）。環境変数 VITE_COGNITO_* から組み立てる。Cognito Hosted UIの独自ログアウトURLを作る cognitoSignOut() もここに定義
auth/AuthTokenSync.tsx	画面には何も描画しない同期用コンポーネント。ログイン状態が変わるたびに api/client.ts の setAuthToken() を呼び、以後のAPIリクエストのAuthorizationヘッダーを最新化する

App.tsx が <AuthProvider> でアプリ全体をラップし、直下に <AuthTokenSync /> をマウントする。各フック・コンポーネントは react-oidc-context の useAuth() を直接呼んでログイン状態を参照する（Reduxのような中央ストアは無く、Contextで完結）。

2.5 純粋ユーティリティ

ファイル	内容	ユニットテスト
utils/score.ts	スコア→色/ラベル変換、評価・人気度アイコン変換、距離ベースのスコア近似再計算	utils/score.test.ts
utils/distance.ts	haversine公式による2点間距離計算	utils/distance.test.ts
types/index.ts	Studio/Recommendation/Favorite/Post/Chat等、全体で共有する型定義	-

3. バックエンド ファイル構成（Lambda）

3.1 API系（backend/lambda/api/handlers.py 1ファイルに集約）

関数	エンドポイント	認証	読み書きするテーブル/バケット
getRecommendationsHandler	GET /recommendations	不要	Recommendations（読）, Studios（読、結合用）
getStudiosHandler	GET /studios	不要	Studios（読）
putStudioImageHandler	PUT /studios/{studioId}/image	必須	Studios（更新）
getPostsHandler	GET /posts	不要	Posts（読）, Users（読、投稿者の表示名を結合）
postPostsHandler	POST /posts	必須	Posts（作成。userIdは_get_user_id(event)で取得しリクエストボディの値は無視）
putPostHandler	PUT /posts/{postId}	必須	Posts（更新。content/imageUrl/ratingのうち指定されたフィールドのみ更新）
deletePostHandler	DELETE /posts/{postId}	必須	Posts（削除。postId単一PKで所有者情報がキーに無いため、削除前にget_itemして自分の投稿か明示チェック。他人の投稿は403）

getFavoritesHandler	GET /favorites	必須	Favorites（読）、Studios（読、結合用）
postFavoritesHandler	POST /favorites	必須	Favorites（作成）
deleteFavoritesHandler	DELETE /favorites/{studioId}	必須	Favorites（削除）
postPresignUploadHandler	POST /uploads/presign	必須	S3 UploadsBucket（署名付きPOSTフォーム発行のみ）
postChatHandler	POST /chat	必須	Chats（読み書き）、Usage（1日あたり利用回数のアトミック加算）、Studios・Recommendations（読、実データグラウンディング用）、S3（画像読み取り）、SSM（Claudeキー）
getChatsHandler	GET /chats	必須	Chats（読、軽量版）
getChatHandler	GET /chats/{chatId}	必須	Chats（読、全メッセージ）
deleteChatHandler	DELETE /chats/{chatId}	必須	Chats（削除）
getMyProfileHandler	GET /me	必須	Users（読）。未登録でも404にせずdisplayName:nullで200を返す
putMyProfileHandler	PUT /me	必須	Users（作成/更新。表示名は1〜30文字でバリデーション）

「認証必須」のエンドポイントは `template.yaml` の `Api` リソースに定義した `CognitoAuthorizer`（Cognito User Pool Authorizer）を通過して初めて呼び出せる。通過後のイベントには `event.requestContext.authorizer.claims.sub` にユーザーの一意なID（Cognitoのsub）が入り、`_get_user_id(event)` ヘルパーがこれを取得する（Favorites/Chatsテーブルは元々userIdがpartition keyの一部のため、他人になりすましてアクセスすること自体ができない設計。PostsテーブルだけpostId単一PKのため上記の通り明示チェックが必要）。

3.2 バッチ系（backend/lambda/batch/）

ファイル	役割	他ファイルとの関係
batch_common.py	DynamoDB/SSM/HTTP GETの共有ヘルパー。 get_user_id/check_and_increment_daily_usage （handlers.pyの同名関数と同じロジックのapi/非依存版、コスト保護レート制限用）	generate_studio_score.py・ discover_studios.py・admin_trigger.py から import される（循環import回避のため切り出し）
generate_studio_score.py	generateStudioScoreBatch本体。口コミ評価・人気度・設備・最寄り駅距離・料金からスコア計算→Claude推薦理由生成→Recommendations保存	batch_common を import。handler冒頭で discover_studios.run_discovery() を呼ぶ（新スタジオ探索も内包）
discover_studios.py	Google Places APIで新規スタジオを探索しStudiosへ追加する中核ロジック（run_discovery）。新規候補ごとに設備・収容人数の推測（Claude）、最寄り駅検索（Places Nearby Search）、公式サイト取得+料金抽出（Place Details→サイト取得→Claude）を行う	batch_common を import。 generate_studio_score.py から呼ばれる他、discoverStudiosBatch Lambdaとしても単独稼働
admin_trigger.py	フロントのボタンから対象バッチをEvent(非同期)invokeする汎用ラッパー。環境変数 RATE_LIMIT_ACTION/RATE_LIMIT_DAILYでコスト保護	環境変数 BATCH_FUNCTION_NAME により、同一コードが adminRunAiBatch と adminRunStudioDiscovery の2つの Lambdaとして再利用されている（レー

	のレート制限を実装（上限超過時はバッチを起動せず429）	ト制限の上限値もこの環境変数で関数ごとに変えている）
--	------------------------------	----------------------------

backend/scripts/backfill_studio_details.py はLambdaではなく、スコアリング項目の 追加（収容人数・最寄り駅距離・料金）以前に発見済みだった既存スタジオへ、これらの フィールドを後追いで埋めるための一回限りのメンテナンススクリプト（python backfill_studio_details.py [--dry-run]）。discover_studios.py の guess_capacity/find_nearest_station_distance_m/fetch_place_website/scrape_price_from_websiteを そのまま再利用し、本番の DynamoDBテーブルを直接更新する。

3.3 SAM関数 ⇔ ハンドラー対応 (infrastructure/template.yaml)

SAM論理ID	Handler指定	トリガー
GenerateStudioScoreBatch	generate_studio_score.handler	EventBridge（毎日AM6:00 JST） ／AdminRunAiBatchFunction経由
DiscoverStudiosBatch	discover_studios.handler	AdminRunStudioDiscoveryFunction 経由のみ（自動スケジュールなし）
AdminRunAiBatchFunction	admin_trigger.handler （BATCH_FUNCTION_NAME=GenerateStudioScoreBatch, RATE_LIMIT_ACTION=ai-batch, RATE_LIMIT_DAILY=3）	POST /admin/run-ai-batch（認証必須）
AdminRunStudioDiscoveryFunction	admin_trigger.handler （BATCH_FUNCTION_NAME=DiscoverStudiosBatch, RATE_LIMIT_ACTION=studio-discovery, RATE_LIMIT_DAILY=10）	POST /admin/run-studio-discovery （認証必須）

Lambda関数名について: API系・バッチ系とも FunctionName は明示指定せず、CloudFormationに自動採番させている。当初は釣行AIアプリと同じ命名で固定していたが、Lambda関数名はアカウント+リージョンで一意という制約に反して既存の釣行AIアプリの 同名関数と衝突し、初回デプロイが失敗した経緯がある（AWS構成・サービス連携.html参照）。

4. DynamoDBテーブル一覧

物理名	論理ID	キー構成	書き込み元
studio-studios	StudiosTable	studioId (PK)	discover_studios.run_discovery, putStudioImageHandler, backfill_studio_details.py（手動）
studio-recommendations	RecommendationsTable	studioId (PK)	generate_studio_score.py（generateStudioScoreBatch実行時のみ）
studio-favorites	FavoritesTable	userId (PK) + studioId (SK)	postFavoritesHandler / deleteFavoritesHandler
studio-posts	PostsTable	postId (PK)	postPostsHandler / putPostHandler / deletePostHandler
studio-chats	ChatsTable	userId (PK) + chatId (SK)	postChatHandler / deleteChatHandler
studio-usage	UsageTable	userId (PK) + dateKey (SK, 例: "ai-batch#2026-07-29")	admin_trigger.py / postChatHandler内の check_and_increment_daily_usage（ADDによるアトミック加算）。TTL（expiresAt）で翌々日ごろ自動削除
studio-users	UsersTable	userId (PK)	putMyProfileHandler（表示名の作成/更新）。getPostsHandlerがscanして投稿者名の結合に使う

S3バケット（studio-search-app-uploads-＜アカウントID＞）はスタジオ写真・投稿写真・チャット添付画像のアップロード先。書き込みは署名付きPOSTフォーム経由のみ、読み取り（GetObject）のみ公開許可。フロント配信用バケット（studio-search-app-frontend-＜アカウントID＞）は別物で、CloudFrontのOrigin Access Control経由でのみ読み取り可能な非公開バケット。

5. データの流れ（代表的なシナリオ）

5.1 「AI分析を実行」を押したとき

```
TopPage.tsx（ボタン）
→ useRecommendations.ts の triggerAiBatch()
→ api/client.ts の runAiBatch()
→ POST /admin/run-ai-batch
→ AdminRunAiBatchFunction（admin_trigger.py）
→ GenerateStudioScoreBatch を非同期 invoke
→ generate_studio_score.py の handler()
→ discover_studios.run_discovery()（新スタジオ探索。新規候補ごとに設備・収容人数推測、
  最寄り駅検索、公式サイト料金抽出を行いStudiosへ追加）
→ Studios全件スキャンスコア計算（最寄り駅距離を使用）→Claude推薦理由生成
→ Recommendations へ保存
→ フロントは GET /recommendations を定期的にポーリングして完了を検知
```

5.2 チャットで写真付きの質問をしたとき

```
ChatInput.tsx（写真選択 + 送信）
→ useChat.ts の send()
→ api/client.ts の getPresignedUploadUrl() → POST /uploads/presign
→ api/client.ts の uploadImageToS3()（S3へ直接POST、バックエンドを経由しない）
→ api/client.ts の sendChatMessage() → POST /chat
→ postChatHandler（handlers.py）
→ S3から画像バイトを取得 → Claudeへマルチモーダル入力として送信
→ Chatsテーブルへ保存 → 応答を同期的に返却
```

5.3 「初心者でも通えるスタジオある？」とAI相談で聞いたとき

```
ChatInput.tsx（現在地共有トグルON + 送信）
→ useChat.ts の send()（lat/lngを含めてsendChatMessage呼び出し）
→ POST /chat
→ postChatHandler（handlers.py）
→ _build_studios_context(lat, lng)がStudios・Recommendationsをscan→結合→
  現在地からの距離（haversine）で近い順にソート→上位N件をテキスト化
→ _generate_chat_reply()のsystem_instructionに上記テキストを埋め込む
  （「データに無いスタジオは作り出さない」という指示も併記）
→ Claudeが実データの中から回答を組み立てる
```

6. 使用サービスとコストの目安

釣行AIアプリと同じAWSアカウント上で運用しているため、AWS側の各サービスは既存分と合算で 無料枠・低ボリューム内に収まる想定。**正確な最新料金は各サービスの公式ページで確認すること**（料金体系は変更されうるため、あくまで目安）。

サービス	課金対象	個人利用規模での目安
Lambda	呼び出し回数・実行時間	無料枠内（月100万回・40万GB秒まで無料）
API Gateway (REST)	APIコール数	この規模ならごく僅か
DynamoDB（7テーブル）	読み書きリクエスト数・ストレージ	無料枠+低ボリュームでごく僅か

S3（静的ホスティング+アップロード）	ストレージ・リクエスト数	数百円未満想定
CloudFront	データ転送量	低ボリュームなら僅か
SSM Parameter Store	Standard利用なら無料	無料
EventBridge（日次バッチ起動）	ルール数・イベント数	無料枠内
Cognito	月間アクティブユーザー数	無料枠内
AWS Budgets	予算数	2つまで無料。今回1つのみ設定
Anthropic Claude API（Haiku）	トークン数（画像添付・料金抽出のHTML抜粋は増加）	最も課金リスクが高い部分 。無料枠は無く完全従量課金だがHaikuは低単価。AI相談・AI分析・新スタジオ探索へのレート制限で抑制済み
Google Places API	リクエスト数（Text Search・Nearby Search・Place Details）	新規スタジオ発見時のみ複数回課金。既存スタジオでは発生しない
Google Maps JavaScript API	読み込み回数	低ボリューム
Cloudflare Workers（ミラー）	リクエスト数	無料枠（1日10万リクエスト）で十分

AWS側は `CostBudget`（AWS Budgets、月\$10・80%/100%到達時にメール通知）で自動監視している。Google Cloud側（Places/Maps）とAnthropic側（Claude）はAWS Budgetsの対象外のため、それぞれのコンソールで別途予算アラート・`spend limit`を手動設定する必要がある（README.mdの「10. 利用料アラート」参照）。

7. 関連ドキュメント

- 企画の背景・釣行AIアプリからの変換経緯: [企画・設計アーカイブ.html](#)
- AWS構成・サービス連携の詳細: [AWS構成・サービス連携.html](#)
- Lambda単位の詳細仕様: [詳細設計.html](#)
- 非技術者向けの全体像: [アプリまるわかりガイド.html](#)